

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Full Stack development MERN Practical

Practical– VIII

Roll No.: T009	Name: Siddhesh Kamble
Class: TYIT	Batch: 01
Date of Assignment: : 30/09/2026	Date/Time of Submission: 30/09/2026

Practical 8: API Creation and Integration

Aim:

To create and integrate a REST API using Express, MongoDB and React.

Explanation

- JavaScript normally works on a browser but with Node JS we can use it on server.
- Express is library runs on nodejs which makes API easy.
- Server.js creates express, allow react to communicate, read JSON data, connect to MongoDB, create API and start server.

A. Create a Simple REST API using Express

Steps:

- Create APIPractical folder and open it in VS Code.
- Run **npm init -y**.

```
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical> npm init -y
Wrote to C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical\package.json:

{
  "name": "apipractical",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

- Install mongoose, express and cors.

```
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical> npm install mongoose express cors
added 88 packages, and audited 89 packages in 5s

30 packages are looking for funding
run `npm fund` for details

found 0 vulnerabilities
```

- Create **server.js**.

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());
mongoose.connect("mongodb://127.0.0.1:27017/college")
  .then(() => console.log("MongoDB Connected"));

const Order = mongoose.model("students", new mongoose.Schema({
  studentName: String,
  rollNo: Number,
  course: String,
  foodItem: String,
  price: Number,
  quantity: Number,
```

```

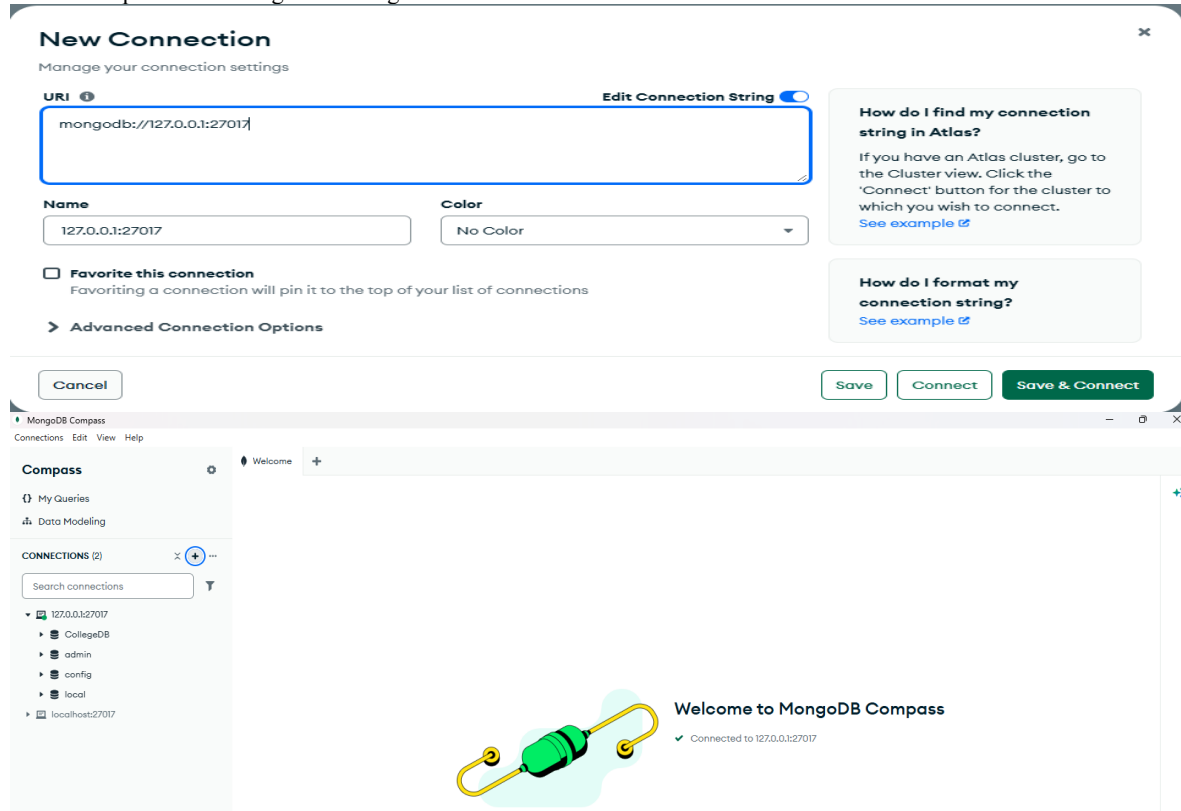
    total: Number
  });
  const food = [
    { name: "Pizza", price: 100 },
    { name: "Veg Burger", price: 60 },
    { name: "Sandwich", price: 50 },
    { name: "Samosa", price: 20 }
  ];
  // GET /food
  app.get("/food", (req, res) => {
    res.json(food);
  });
  // POST /orders
  app.post("/orders", async (req, res) => {
    const { studentName, rollNo, course, foodItem, quantity } = req.body;

    const item = food.find(f => f.name === foodItem);
    const total = item.price * quantity;

    const order = new Order({
      studentName,
      rollNo,
      course,
      foodItem,
      price: item.price,
      quantity,
      total
    });
    await order.save();
    res.json(order);
  });
  // GET /orders
  app.get("/orders", async (req, res) => {
    const orders = await Order.find();
    res.json(orders);
  });
  app.listen(5000, () => console.log("Server running on 5000"));

```

5. Connect Express with MongoDB CollegeDB.



The image shows the MongoDB Compass interface. At the top, the 'New Connection' dialog is open, allowing the user to manage connection settings. The 'URI' field is populated with 'mongodb://127.0.0.1:27017'. Below this, the 'Name' field is set to '127.0.0.1:27017' and the 'Color' is set to 'No Color'. There are checkboxes for 'Favorite this connection' and 'Advanced Connection Options'. On the right side of the dialog, there are two informational boxes: 'How do I find my connection string in Atlas?' and 'How do I format my connection string?'. At the bottom of the dialog are buttons for 'Cancel', 'Save', 'Connect', and 'Save & Connect'.

Below the dialog, the main MongoDB Compass interface is visible. The 'CONNECTIONS (2)' list on the left shows the connection '127.0.0.1:27017' with a sub-entry 'CollegeDB'. The main area displays a 'Welcome to MongoDB Compass' message with a status indicator 'Connected to 127.0.0.1:27017' and a decorative graphic of a green battery with yellow wires.

6. Create a simple GET route /.
7. Run node server.js.

```
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical> node server.js
Server is running on http://localhost:3000
MongoDB Connected Successfully
```

- 8.
9. Open http://localhost:3000.

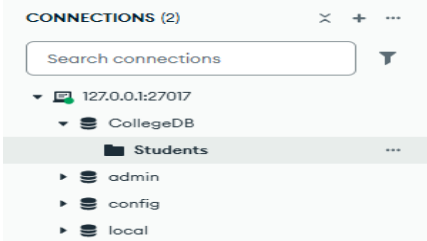
Output:



B. Implement GET API

Steps:

1. Create CollegeDB database and students collection in MongoDB.



2. Insert student records.



3. Add GET /students API in server.js.

In server.js, add the following above **app.listen()**

```
app.get("/students", async (req, res) => {
  try {
    const students = await mongoose.connection.db
      .collection("students")
      .find()
      .toArray();
    res.json(students);
  }
  catch (error) {
    res.status(500).json({
      message: "Error fetching students"
    });
  }
});
```

4. Run the server.
5. Open http://localhost:3000/students.

Output:

```

[
  {
    "_id": "6ab38324b44a102cd2fd9b20",
    "rollNo": 1,
    "name": "Siddhesh Kamble",
    "course": "BSc IT",
    "marks": 85
  },
  {
    "_id": "6ab38343b44a102cd2fd9b22",
    "rollNo": 2,
    "name": "Rahul Patil",
    "course": "BSc IT",
    "marks": 78
  }
]

```

C. Fetch API Data in React

Steps:

1. Create a React project using Vite.

```

PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical> npm create vite@latest my-react
Need to install the following packages:
create-vite@0.2.1
Ok to proceed? (y) Y

> apipractical@1.0.0 npx
> create-vite my-react

Select a framework:
  React

Select a variant:
  JavaScript

Which linter to use?
  ESLint

Install with npm and start now?
  Yes

Scaffolding project in C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical\my-react...

Installing dependencies with npm...

added 143 packages, and audited 144 packages in 20s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

Starting dev server...

> my-react@0.0.0 dev
> vite

VITE v8.3.0 ready in 2308 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help

PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical> cd my-react
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical\my-react> npm install

up to date, audited 144 packages in 725ms

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

2. Install Axios.

```

PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\APIPractical\my-react> npm install axios

added 25 packages, and audited 169 packages in 4s

37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

3. Add API request in App.jsx.

App.jsx :

```

import { useEffect, useState } from "react";
import axios from "axios";

function App() {

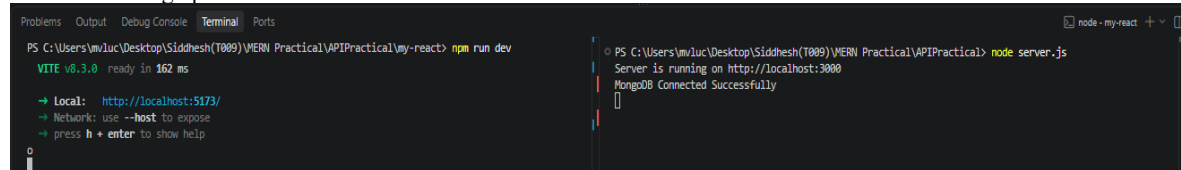
```

```

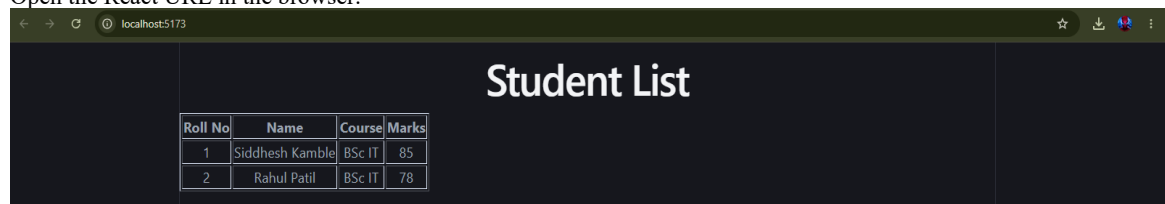
const [students, setStudents] = useState([]);
useEffect(() => {
  axios.get("http://localhost:3000/students")
    .then((response) => {
      setStudents(response.data);
    })
    .catch((error) => {
      console.log(error);
    });
}, []);
return (
  <div>
    <h1>Student List</h1>
    <table border="1">
      <thead>
        <tr>
          <th>Roll No</th>
          <th>Name</th>
          <th>Course</th>
          <th>Marks</th>
        </tr>
      </thead>
      <tbody>
        {students.map((student) => (
          <tr key={student._id}>
            <td>{student.rollNo}</td>
            <td>{student.name}</td>
            <td>{student.course}</td>
            <td>{student.marks}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
}
export default App;

```

- Start Express server using node server.js.
- Start React using npm run dev.



- Open the React URL in the browser.



D. Send Data using POST Request

Steps:

- Add POST /students API in server.js.

Add this code **above app.listen()**

```

app.post("/students", async (req, res) => {
  try {
    const student = req.body;
    await mongoose.connection.db
      .collection("students")
      .insertOne(student);
    res.json({
      message: "Student added successfully",
    });
  } catch (error) {
    console.log(error);
  }
});

```

```

        student: student
      });
    }
    catch (error) {
      res.status(500).json({
        message: "Error adding student"
      });
    }
  }
});

```

App.jsx :

```

import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [students, setStudents] = useState([]);
  const [rollNo, setRollNo] = useState("");
  const [name, setName] = useState("");
  const [course, setCourse] = useState("");
  const [marks, setMarks] = useState("");
  // Get students
  const getStudents = () => {
    axios.get("http://localhost:3000/students")
      .then((response) => {
        setStudents(response.data);
      })
      .catch((error) => {
        console.log(error);
      });
  };

  useEffect(() => {
    getStudents();
  }, []);

  // Add student
  const addStudent = () => {
    const newStudent = {
      rollNo: Number(rollNo),
      name: name,
      course: course,
      marks: Number(marks)
    };
    axios.post("http://localhost:3000/students", newStudent)
      .then((response) => {
        alert(response.data.message);
        setRollNo("");
        setName("");
        setCourse("");
        setMarks("");
        getStudents();
      })
      .catch((error) => {
        console.log(error);
      });
  };

  return (
    <div>
      <h1>Student Management System</h1>
      <h2>Add Student</h2>
      <input
        type="number"
        placeholder="Roll No"
        value={rollNo}
        onChange={(e) => setRollNo(e.target.value)}
      />
      <br /><br />
      <input
        type="text"
        placeholder="Student Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <br /><br />
      <input
        type="text"
        placeholder="Course"
        value={course}

```

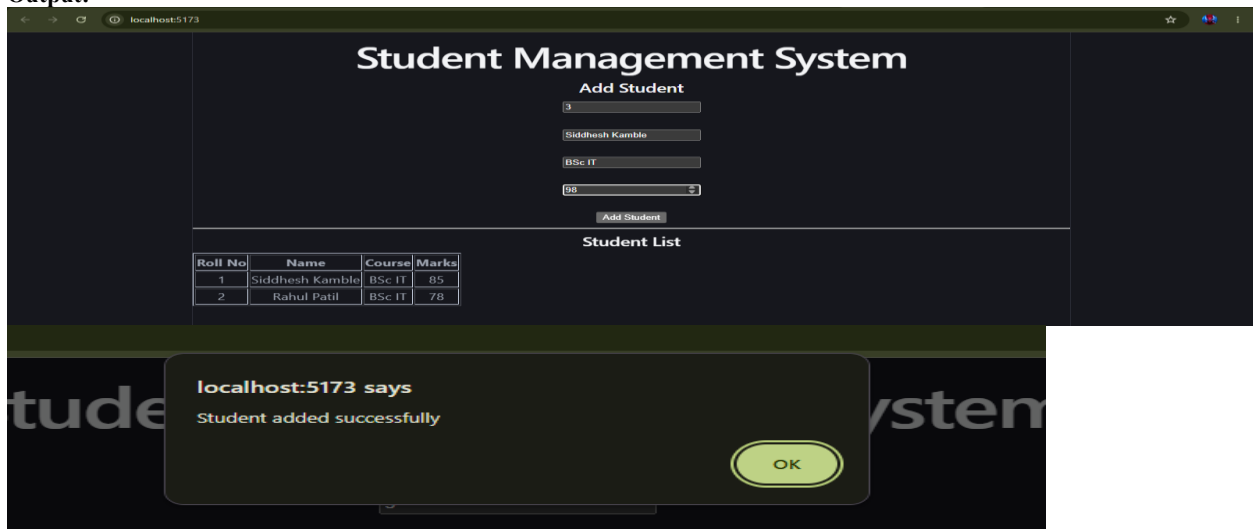
```

        onChange={ (e) => setCourse(e.target.value) }
      />
    <br /><br />
    <input
      type="number"
      placeholder="Marks"
      value={marks}
      onChange={ (e) => setMarks(e.target.value) }
    />
    <br /><br />
    <button onClick={addStudent}>
      Add Student
    </button>
    <hr />
    <h2>Student List</h2>
    <table border="1">
      <thead>
        <tr>
          <th>Roll No</th>
          <th>Name</th>
          <th>Course</th>
          <th>Marks</th>
        </tr>
      </thead>
      <tbody>
        {students.map((student) => (
          <tr key={student._id}>
            <td>{student.rollNo}</td>
            <td>{student.name}</td>
            <td>{student.course}</td>
            <td>{student.marks}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
}
export default App;

```

2. Create student input form in React.
3. Enter Roll No, Name, Course and Marks.
4. Click **Add Student**.
5. Axios sends the data to Express.
6. Express stores the data in MongoDB.
7. Student list is refreshed.

Output:



Name : Siddhesh Kamble

Roll No. : T009

Student List

Roll No	Name	Course	Marks
1	Siddhesh Kamble	BSc IT	85
2	Rahul Patil	BSc IT	78
3	Siddhesh Kamble	BSc IT	98

Result

The REST API was successfully created using Express and MongoDB, and integrated with a React application using GET and POST requests.